



République Tunisienne
Ministère de
l'Enseignement Supérieur
de la Recherche Scientifique
Université de Carthage



École Nationale d'Ingénieurs de Carthage

Rapport du Projet

Programmation C : Jeu MOTUS

Présenté par:

Ben Rached Ghofrane

Cherni Soulaïma

Cherif Roua

Supervisé par : Mme Loussaïef Imen

Département Génie Electrique

1ère année Génie des Systèmes Infotroniques

Remerciement

« Nous tenons à exprimer notre profonde gratitude envers l'ENICARthage pour fournir un cadre éducatif exceptionnel.

Nos remerciements vont tout particulièrement à Mme Loussaief Imen, notre superviseure, dont l'orientation experte a été cruciale pour l'achèvement de ce rapport.

Nous souhaitons également remercier l'ensemble du personnel pédagogique et nos collègues pour leur partage de connaissances et leur soutien constant.

Merci à l'ENICARthage pour cette précieuse opportunité d'apprentissage. »

Tableau de Matières

Introduction générale	1-4
Chapitre 1 :Contextualisation et Infrastructure Technique.....	1-5
Introduction :	1-5
1.Contexte du projet :	1-5
2.Langage C :	1-5
3.Code blocks :	1-5
4.Bibliothèques :	1-6
5.Principe du jeu.....	1-7
Conclusion.....	1-9
Chapitre 2 :Analyse Détaillée des Fonctions du Code	2-11
Introduction	2-11
1.Fonctions principales :	2-11
a.La fonction choisirNiveau :	2-11
b.La fonction choisirThemeFichier :	2-13
c.La fonction jouerSolo :	2-14
d.La fonction jouerDeuxJoueurs :	2-18
e.La fonction admin :	2-21
f.La fonction afficheRésultat :	2-22
g.La fonction supprimerMot.....	2-23
h.La fonction main	2-24
Conclusion.....	2-26
Conclusion générale	2-27
Bibliographie	2-28

Tableau des figures

Figure 1 codeBlocks logo.....	1-6
Figure 2 Appel de la bibliothèque stdio.h	1-6
Figure 3 Appel de la bibliothèque Stdlib.h.....	1-7
Figure 4 Appel de la bibliothèque string. h	1-7
Figure 5 Appel de la bibliothèque time. h	1-7
Figure 6 diagramme de motus	1-7
Figure 7 diagramme d'admin	1-8
Figure 8 digramme de jouer solo.....	1-8
Figure 9 diagramme jouer a deux.....	1-9
Figure 10 partie 1 fonction choisierNiveau	2-11
Figure 11 Partie 2 du la fonction choisierNiveau	2-12
Figure 12 Partie 3 du la fonction choisierNiveau	2-12
Figure 13 Partie 4 du la fonction choisierNiveau	2-13
Figure 14 la fonction choisierThemeFichier	2-13
Figure 15 thèmes Tunisies.....	2-14
Figure 16 thème autre.....	2-14
Figure 17 partie 1 de la fonction jouerSolo	2-15
Figure 18 partie 2 de la fonction jouerSolo	2-15
Figure 19 partie 3 de la fonction joue Solo	2-16
Figure 20 partie 4 de la fonction jouerSolo	2-16
Figure 21 partie 5 de la fonction de jouerSolo	2-17
Figure 22 partie 6 de la fonction jouerSolo	2-17
Figure 23 partie 7 de la fonction de jouerSolo	2-18
Figure 24 partie 1 de la fonction jouerDeuxJoueurs.....	2-18
Figure 25 partie 2 de la fonction jouerDeuxJoueurs.....	2-19
Figure 26 partie 3 de la fonction jouerDeuxJoueurs.....	2-19
Figure 27 partie 4 de la fonction jouerDeuxJoueurs.....	2-20
Figure 28 partie 5 de la fonction jouerDeuxJoueurs.....	2-20
Figure 29 partie 1 de la fonction admin.....	2-21
Figure 30 partie 2 de la fonction admin.....	2-22
Figure 31 fonction afficheResultat	2-23
Figure 32 partie 1 de La fonction supprimerMot	2-23
Figure 33 partie 2 de La fonction supprimerMot	2-24
Figure 34 partie 1 de La fonction main	2-25
Figure 35 partie 2 de La fonction main	2-25
Figure 36 partie 3 de La fonction main	2-25

Introduction générale

Les jeux, avec leur capacité innée à captiver les joueurs, ont toujours constitué un véritable phénomène culturel. Leur attrait massif témoigne de l'engouement que suscitent ces expériences ludiques. C'est dans ce contexte dynamique que s'inscrit notre projet, visant à transcender la simple notion de divertissement. Nous nous sommes inspirés de la passion que les joueurs vouent aux jeux pour créer un jeu Motus unique, unissant le plaisir ludique à une dimension particulière axée sur la concentration.

Dans ce rapport, nous entreprenons un voyage à travers le développement de notre jeu Motus. Le chapitre initial jettera les bases de notre exploration en exposant le contexte du projet, en dévoilant le principe de fonctionnement, les bibliothèques utilisées, le langage de programmation (C), et le logiciel (Code : Blocks) qui ont contribué à la conception du jeu.

Le chapitre suivant approfondira notre compréhension en analysant chaque fonction du code, fournissant des explications détaillées pour éclairer leur rôle et leur interaction. Cette plongée dans les détails technologiques établira une compréhension solide de la structure sous-jacente de notre jeu.

Chapitre 1 : Contextualisation et Infrastructure Technique

Introduction :

Dans ce chapitre, nous explorerons le contexte du projet, le principe de fonctionnement, les bibliothèques intégrées dans notre code, le langage de programmation utilisé, ainsi que le logiciel employé.

1.Contexte du projet :

Le projet consiste à développer un jeu Motus en langage C avec deux objectifs principaux. Premièrement, créer une version tunisienne du jeu en intégrant des thèmes culturels locaux et en utilisant la langue arabe tunisienne. Deuxièmement, offrir une expérience visant à aider les utilisateurs à développer leur concentration, tout en assurant un divertissement de qualité. Le projet vise à fusionner le plaisir du jeu avec la richesse culturelle de la Tunisie, tout en contribuant au développement cognitif des utilisateurs.

2.Langage C :

Le choix du langage de programmation C pour le développement du jeu Motus repose sur plusieurs considérations cruciales :

- ❖ Performance : Le langage C est réputé pour sa performance. Il offre un contrôle direct sur la mémoire et une exécution rapide, ce qui le rend adapté à des applications où la performance est cruciale.
- ❖ Héritage et polyvalence :Le C est un langage de programmation ancien avec un riche héritage. Il est utilisé dans le développement de systèmes d'exploitation, de logiciels embarqués, et de nombreuses autres applications.
- ❖ Large communauté et ressources : Le C a une communauté établie et de nombreuses ressources d'apprentissage disponibles, ce qui peut être un avantage lors du développement d'un projet.

3.Code blocks :

Le choix de Code Blocks comme environnement de développement pour le jeu Motus peut découler de plusieurs raisons. Voici quelques arguments qui pourraient justifier ce choix :

- ❖ **Gratuit et Open sources :** Code Blocks est un logiciel gratuit et open source, ce qui signifie qu'il est accessible à tous sans coût supplémentaire. Cela peut être un critère important, en particulier pour les projets avec des contraintes budgétaires.
- ❖ **Compatibilité multiplateforme :** Code Blocks est compatible avec plusieurs systèmes d'exploitation, y compris Windows, Linux et MacOS. Cette compatibilité multiplateforme assure que le développement peut être effectué sur différentes machines sans nécessiter des ajustements majeurs.
- ❖ **Fonctionnalités étendues :** Code Blocks propose un ensemble complet de fonctionnalités, notamment un éditeur de texte avancé, un débogueur intégré, la gestion de projets, et la possibilité d'ajouter des plugins pour étendre ses capacités.

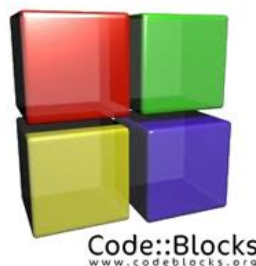


Figure 1 codeBlocks logo

4. Bibliothèques :

Pour le développement d'un jeu Motus en langage C, nous aurons besoin de certaines bibliothèques pour gérer les divers aspects du jeu :

- **<Stdio.h>** : Cette bibliothèque standard est essentielle pour les opérations d'entrée/sortie standard, notamment pour l'affichage de messages à l'utilisateur et la saisie de données depuis le clavier.



Figure 2 Appel de la bibliothèque stdio.h

- **<Stdlib.h>** : Cette bibliothèque fournit plusieurs fonctions pour effectuer plusieurs opérations liées à la gestion de la mémoire dynamique et d'autres fonctions utilitaires.

```
c Copy code
#include <stdlib.h>
```

Figure 3 Appel de la bibliothèque Stdlib.h

- **<String. h>** : cette bibliothèque permet de fournir les opérations de manipulation de chaînes de caractères, comme la comparaison et la copie.

```
c Copy code
#include <string.h>
```

Figure 4 Appel de la bibliothèque string. h

- **<time. h>** : Cette bibliothèque fournit des fonctions pour travailler avec le temps et la date.

```
c Copy code
#include <time.h>
```

Figure 5 Appel de la bibliothèque time. h

5.Principe du jeu

Le jeu de Motus est une application impliquant plusieurs acteurs : un administrateur et un ou deux joueurs.

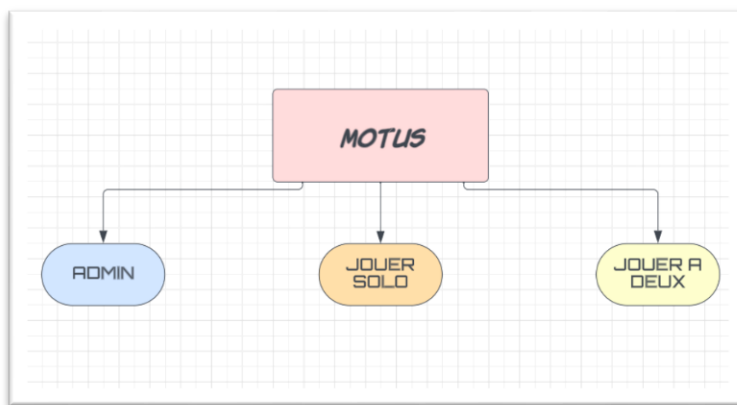


Figure 6 diagramme du motus

L'administrateur est responsable à la gestion des données, telle que l'ajout de nouveaux mots, ou la suppression des mots déjà existants.

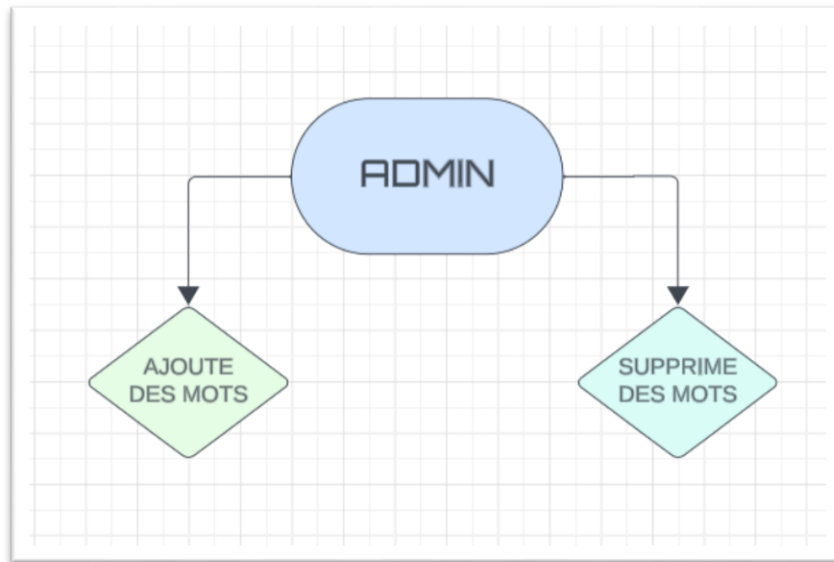


Figure 7 diagramme d'admin

Dans le mode solo, le joueur a la possibilité de sélectionner le niveau de difficulté et de décider du nombre de parties auxquelles il souhaite participer. Son objectif est de deviner un mot mystère en utilisant un maximum de quatre essais. S'il réussit à deviner le mot dans cette limite, il remporte la manche et obtient un score. En revanche, s'il épuise ses essais sans trouver le mot, il perd la partie.

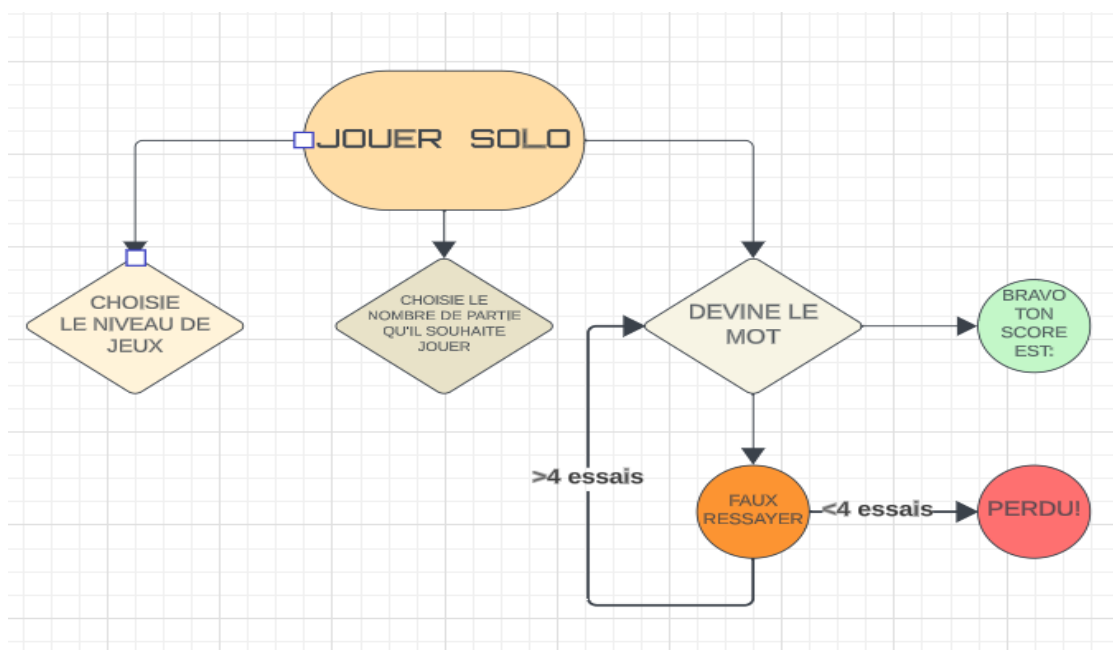


Figure 8 digramme jouer solo

En mode duo, le joueur 1 sélectionne un niveau de difficulté et détermine le nombre de parties auxquelles ils souhaitent jouer à deux. Les joueurs alternent pour essayer de deviner le mot en utilisant un maximum de quatre essais chacun. Si l'un des joueurs devine correctement le mot, il remporte la partie et obtient le score gagnant pour cette manche. Si aucun des deux joueurs ne devine correctement, les deux perdent la partie.

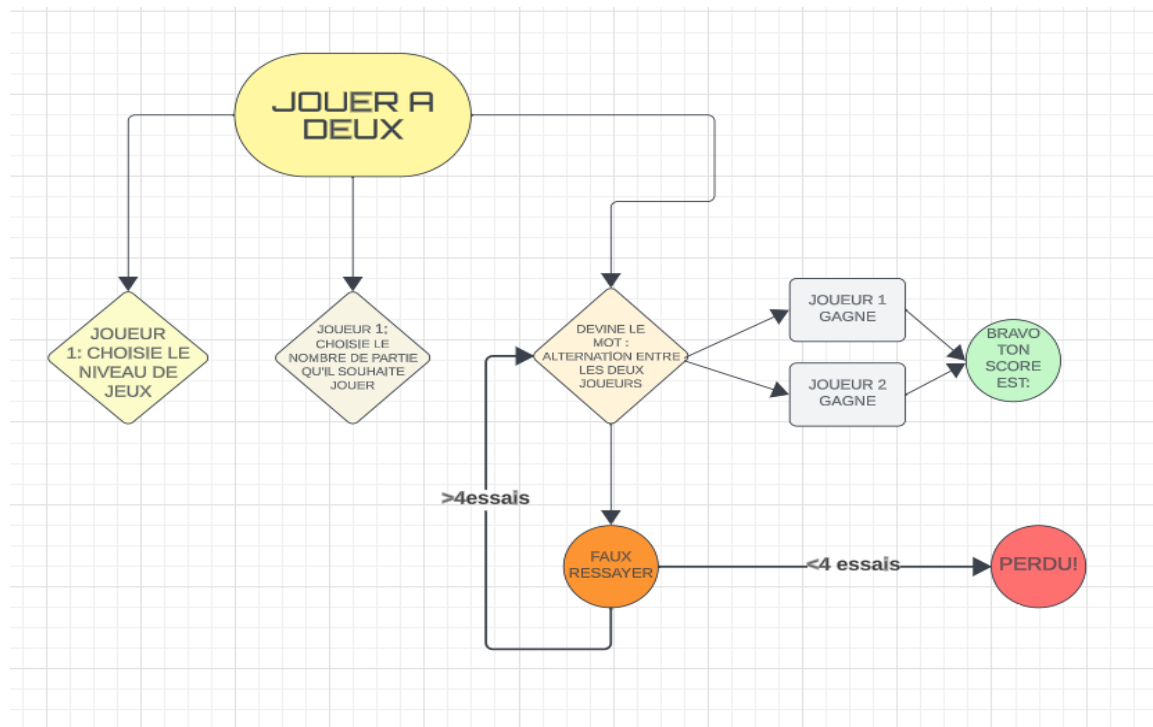


Figure 9 diagramme jouer a deux

Conclusion

Dans ce chapitre introductif, nous avons établi le contexte du projet, exposé le principe de fonctionnement du jeu Motus, identifié les bibliothèques intégrées dans notre code, spécifié le langage de programmation utilisé (langage C), et présenté le logiciel employé (Code :Blocks). Ces informations constituent une base solide pour la compréhension ultérieure du développement du jeu Motus avec ses spécificités tunisiennes.

Chapitre 2 :Analyse Détaillée des Fonctions du Code

Introduction

Dans ce chapitre, nous procéderons à une analyse détaillée de chaque fonction présente dans notre code, en fournissant des explications approfondies pour une compréhension détaillée

1.Fonctions principales :

Dans le cadre du développement du jeu Motus en langage C, la création d'un code bien structuré et modulaire est cruciale. Les fonctions jouent un rôle central dans cette approche, permettant une organisation claire du code, une réutilisation efficace, et une maintenance simplifiée.

a.La fonction choisirNiveau :

Semble destinée à permettre au joueur de choisir le niveau de difficulté pour plusieurs parties d'un jeu.

Voici le code commenté et expliqué :

```
#ifndef NIVEAU_H
#define NIVEAU_H

#include <stdio.h>

// Fonction pour choisir le niveau de difficulté
void choisirNiveau(FILE *fichier, int *longueurMin, int *longueurMax) {
    int nombreDeParties;

    // Demande du nombre de parties à jouer
    printf("Combien de parties souhaitez-vous jouer ? : ");
    scanf("%d", &nombreDeParties);

    for (int i = 0; i < nombreDeParties; i++) {
        int NbMot = 0;
        char mot[10];

        // Compte le nombre de mots dans le fichier
        while (fscanf(fichier, "%s", mot) != EOF) {
            NbMot++;
        }
    }
}
```

Figure 10 partie 1 fonction choisirNiveau

```

rewind(fichier);
// Si le fichier est vide, ajoute des mots par défaut
if (NbMot == 0) {
    fclose(fichier);
    fichier = fopen("my_file.txt", "a");
    if (fichier != NULL) {
        remplirFichierAvecMotsParDefaut(fichier);
        fclose(fichier);
    }
    fichier = fopen("my_file.txt", "r");
}

int niveau;
// Le joueur choisit le niveau (0 pour facile, 1 pour normal, 2 pour difficile)
printf("Joueur 1, choisissez le niveau (0 pour facile, 1 pour normal, 2 pour difficile) : ");
scanf("%d", &niveau);

int longueurMin, longueurMax;
// Configuration des longueurs de mot en fonction du niveau choisi
if (niveau == 0) {
    longueurMin = 3;

```

Figure 11 Partie 2 du la fonction choisirNiveau

```

    longueurMin = 3;
    longueurMax = 4;
} else if (niveau == 1) {
    longueurMin = 5;
    longueurMax = 6;
} else if (niveau == 2) {
    longueurMin = 7;
    longueurMax = 8;
} else {
    printf("Choix de niveau non valide. Veuillez choisir 0 pour facile, 1 pour normal, ou 2 pour difficile.\n");
    return; // Retourne si le choix de niveau n'est pas valide
}

int motsFacile = 0, motsNormal = 0, motsDifficile = 0;
// Compte le nombre de mots dans chaque catégorie de niveau
rewind(fichier);
while (fscanf(fichier, "%s", mot) != EOF) {
    int motLength = strlen(mot);

    if (motLength >= 3 && motLength <= 4) {
        motsFacile++;
    } else if (motLength >= 5 && motLength <= 6) {

```

Figure 12 Partie 3 du la fonction choisirNiveau

```

        motsNormal++;
    } else if (motLength >= 7 && motLength <= 8) {
        motsDifficile++;
    }
}
// Vérifie s'il y a suffisamment de mots de chaque niveau
if (motsFacile == 0 || motsNormal == 0 || motsDifficile == 0) {
    printf("Le fichier ne contient pas suffisamment de mots de chaque niveau.
    | L'administrateur doit ajouter des mots de chaque niveau.\n");
    return; // Retourne si le fichier ne contient pas suffisamment de mots de chaque niveau
}
}
#endif // NIVEAU_H

```

Figure 13 Partie 4 de la fonction choisirNiveau

b. La fonction choisirThemeFichier :

Elle semble être conçue pour sélectionner un fichier de thème en fonction d'un numéro de thème fourni en argument.

- Cette fonction prend en paramètre un numéro de thème (thème) qui est utilisé dans une instruction `switch` pour déterminer le fichier de thème associé.
- Chaque case de l'instruction `switch` correspond à un numéro de thème spécifique et renvoie le fichier associé.
- Si le numéro de thème ne correspond à aucun des cas spécifiés (default), la fonction renvoie le fichier par défaut "my_file.txt".
- La fonction retourne un pointeur vers une chaîne de caractères (const char *) qui est le nom du fichier de thème sélectionné.

Voici le code commenté et expliqué :

```

#ifndef theme_h
#define theme_h
// Fonction pour choisir un fichier de thème en fonction d'un numéro de thème
const char *choisirThemeFichier(int theme) {
    switch (theme) {
        // Si le thème est 1, retourne le fichier "theme_choufli_bal.txt"
        case 1:
            return "theme_choufli_bal.txt";
        // Si le thème est 2, retourne le fichier "theme_labsa_tounsiva.txt"
        case 2:
            return "theme_labsa_tounsiva.txt";
        // Si le thème est 3, retourne le fichier "theme_mekla_tounsiva.txt"
        case 3:
            return "theme_mekla_tounsiva.txt";
        // Si le numéro de thème n'est pas reconnu, retourne le fichier par défaut "my_file.txt"
        default:
            return "my_file.txt";
    }
}
#endif // theme_h

```

Figure 14 la fonction choisirThemeFichier

Ainsi nos thèmes sont :

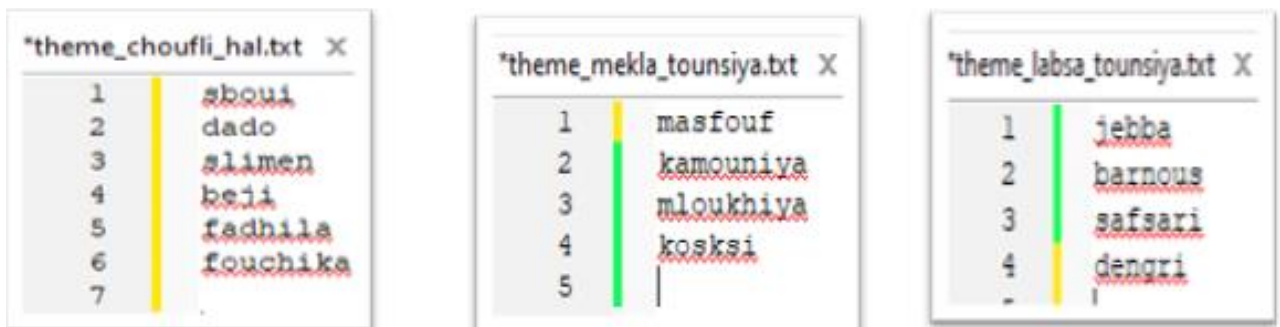


Figure 15 thèmes Tunisies

Sinon si le joueur ou l'administrateur ne veut pas jouer dans un theme specifique, la fonction renvoie le fichier par défaut "my_file.txt" :



Figure 16 thème autre

c. La fonction jouerSolo :

Gère le déroulement de plusieurs parties à un seul joueur en solo. Elle initialise le générateur de nombres aléatoires, demande le nombre de parties, puis itère sur chaque partie.

```

#define SOLO_H
#include <stdio.h> // Inclusion des bibliothèques nécessaires
#include <stdlib.h>
#include <string.h>
#include "theme.h" // Inclusion du fichier d'en-tête externe "theme.h"
// Fonction pour jouer plusieurs parties en solo
// Elle demande le nombre de parties et itère sur chacune d'entre elles
void jouerSolo(FILE *fichier, int *scoreJoueur1, int theme) {
    // Déclaration des variables
    int nombreDeParties;
    // Demande du nombre de parties à jouer
    printf("Combien de parties souhaitez-vous jouer en solo ? : ");
    scanf("%d", &nombreDeParties);
    // Boucle à travers chaque partie
    for (int i = 0; i < nombreDeParties; i++) {
        int NbMot = 0;
        char mot[10];
        // Compte le nombre de mots dans le fichier
        while (fscanf(fichier, "%s", mot) != EOF) {
            NbMot++;
        }
        rewind(fichier); // ramener au début du fichier.
        // Si le fichier est vide, ajoute des mots par défaut
        if (NbMot == 0) {

```

Figure 17 partie 1 de la fonction jouerSolo

```

        char mot[10];
        // Compte le nombre de mots dans le fichier
        while (fscanf(fichier, "%s", mot) != EOF) {
            NbMot++;
        }
        rewind(fichier); // ramener au début du fichier.
        // Si le fichier est vide, ajoute des mots par défaut
        if (NbMot == 0) {
            fclose(fichier);
            fichier = fopen("my_file.txt", "a");
            if (fichier != NULL) {
                remplirFichierAvecMotsParDefaut(fichier); // Fonction externe pour remplir
                                                            // le fichier vide
            }
            fclose(fichier);
            fichier = fopen("my_file.txt", "r");
        }
    }

    int niveau;
    // Le joueur choisit le niveau (0 pour facile, 1 pour normal, 2 pour difficile)
    printf("Joueur 1, choisissez le niveau (0 pour facile, 1 pour normal, 2 pour difficile) : ");
    scanf("%d", &niveau);
    int longueurMin, longueurMax;
    // Configuration des longueurs de mot en fonction du niveau choisi
    if (niveau == 0) {

```

Figure 18 partie 2 de la fonction jouerSolo


```

    longueurMin = 3;
    longueurMax = 4;
} else if (niveau == 1) {
    longueurMin = 5;
    longueurMax = 6;
} else if (niveau == 2) {
    longueurMin = 7;
    longueurMax = 8;
} else {
    printf("Choix | non valide.Veuillez choisir 0 pour facile,1pour normal,2pour difficile.\n");
    return; // Retourne si le choix de niveau n'est pas valide
}
int motsFacile = 0, motsNormal = 0, motsDifficile = 0;
// Compte le nombre de mots dans chaque catégorie de niveau
rewind(fichier);
while (fscanf(fichier, "%s", mot) != EOF) {
    int motLength = strlen(mot);
    if (motLength >= 3 && motLength <= 4) {
        motsFacile++;
    } else if (motLength >= 5 && motLength <= 6) {
        motsNormal++;
    } else if (motLength >= 7 && motLength <= 8) {
        motsDifficile++;
    }
}
}

```

Figure 19 partie 3 de la fonction joue Solo

```

// Vérifie s'il y a suffisamment de mots de chaque niveau
if (motsFacile == 0 || motsNormal == 0 || motsDifficile == 0) {
    printf("Le fichier ne contient pas suffisamment de mots de chaque niveau.\n");
    L'administrateur doit ajouter des mots de chaque niveau.\n";
    return; // Retourne si le fichier ne contient pas suffisamment de mots de chaque niveau
}
do {
    // Choix aléatoire d'un mot dans le fichier
    rewind(fichier);
    int indiceAleatoire = rand() % NbMot;
    for (int i = 0; i <= indiceAleatoire; i++) {
        fscanf(fichier, "%s", mot); // Cela permet de sélectionner un mot au hasard dans le fichier,
        // offrant ainsi une certaine variabilité dans le choix des mots à deviner dans le jeu.
    }
} while (strlen(mot) < longueurMin || strlen(mot) > longueurMax);

char tab2[50]; // Déclaration d'un tableau de caractères pour stocker une copie du mot à deviner
strcpy(tab2, mot); // Copie du mot original dans le tableau tab2
int LONGUEUR_MOT = strlen(mot); // Calcul de la longueur du mot à deviner
char *motSecret = tab2; // Déclaration d'un pointeur vers le tableau contenant le mot à deviner
int essais = 0; // Initialisation du compteur d'essais du joueur
char lettreDevinee[LONGUEUR_MOT]; // Déclaration d'un tableau pour stocker les lettres devinées
memset(lettreDevinee, 0, sizeof(lettreDevinee)); // Initialisation du tableau lettreDevinee avec des zéros
printf("\nJoueur 1, voici le mot à deviner : %c", motSecret[0]); // Affichage de la première lettre du mot à deviner
for (int f = 1; f < LONGUEUR_MOT; f++) {
    printf("-"); // Affichage de tirets pour les lettres restantes du mot à deviner
}

```

Figure 20 partie 4 de la fonction jouerSolo

```

char lettreDevinee1[LONGUEUR_MOT1];
memset(lettreDevinee1, 0, sizeof(lettreDevinee1));

    printf("\n Joueur, voici le mot à deviner :%c", motSecret1[0] );
    for (int f = 1; f < LONGUEUR_MOT1; f++) {
        printf("-");
    }

while (1) {
    if (essais1 > NB_ESSAIS_MAX) { //si il atteint le nombre d'essais max
        printf("\n fin zouz khasriin : ( !!!\n"); //message qu'il ont perdu

        break; //break: fin de jeu pas besoin de continuer
    }
    char motJoueur1[10];
    printf("\nJoueur 1, entrez un mot de %d : ", LONGUEUR_MOT1);
    scanf("%s", motJoueur1);

```

Figure 21 partie 5 de la fonction de jouerSolo

```

if (strlen(motJoueur) != LONGUEUR_MOT) {
    printf("Le mot doit avoir %d lettres.\n", LONGUEUR_MOT); // Vérification de la longueur du mot proposé
    continue; // Reprise de la boucle si la longueur n'est pas correcte
}
essais++; // Incrémentation du compteur d'essais
int lettresCorrectes = 0; // Initialisation du compteur de lettres correctes
// Comparaison des lettres du mot proposé avec le mot secret
for (int f = 0; f < LONGUEUR_MOT; f++) {
    if (motJoueur[f] == motSecret[f]) {
        lettreDevinee[f] = motSecret[f]; // Stockage de la lettre correcte dans le tableau lettreDevinee
        lettresCorrectes++; // Incrémentation du compteur de lettres correctes
    }
}
afficherResultat(motJoueur, motSecret); // Affichage du résultat de la tentative du joueur
if (lettresCorrectes == LONGUEUR_MOT) {
    printf("Mabrouk 🍀 , Joueur 1 a deviné le mot en %d essais : %s\n", essais, motSecret);
    *scoreJoueur1 += essais; // Mise à jour du score du joueur en cas de réussite
    supprimerMot(choisirThemeFichier(theme), motSecret); // Suppression du mot deviné du fichier
    printf("Score après la partie %d - Joueur 1 (Solo) : %d\n", i + 1, essais * 10); // Affichage du score
    break; // Sortie de la boucle après avoir deviné le mot
} else {
    printf("Le mot ");
    for (int f = 0; f < LONGUEUR_MOT; f++) {

```

Figure 22 partie 6 de la fonction jouerSolo

```

        if (lettreDevinee[i] != '\0') {
            printf("%c", lettreDevinee[i]); // Affichage des lettres correctes devinées jusqu'à présent
        } else {
            printf("-");
        }
    }
    printf(" n'est pas le mot secret.\n");
}
}}}
#endif // SOLO.h

```

Figure 23 partie 7 de la fonction de jouerSolo

d. La fonction *jouerDeuxJoueurs* :

Gère le déroulement de plusieurs parties à deux joueurs. Elle initialise le générateur de nombres aléatoires, demande le nombre de parties, puis itère sur chaque partie.

En effet, la fonction *jouerDeuxJoueurs* est essentiellement similaire à celle *jouerSolo* au niveau de choisir le nombre de parties, le niveau ..., avec quelques différences distinctes :

```

#include <stdio.h>
#define duo_h
#include <stdio.h>
void jouerDeuxJoueurs(FILE *fichier, int *scoreJoueur1, int *scoreJoueur2, int theme) {
    srand(time(0));
    int nombreDeParties;
    printf("\n*****\n");
    printf("Combien de parties souhaitez-vous jouer à deux ? : ");
    scanf("%d", &nombreDeParties);
    rewind(fichier);
    for (int i = 0; i < nombreDeParties; i++) {
        int NbMotl = 0;
        char motl[10];
        while (fscanf(fichier, "%s", motl) != EOF) {
            NbMotl++;
        }
        rewind(fichier);
        if (NbMotl == 0) {
            fclose(fichier);
            fichier = fopen("my_file.txt", "a");
            if (fichier != NULL) {
                remplirFichierAvecMotsParDefaut(fichier);
                fclose(fichier);
            }
            fichier = fopen("my_file.txt", "r");
        }
        int niveau;
        printf("Joueur 1, choisissez le niveau\n 0 pour facile\n 1 pour normal\n 2 pour difficile\n Votre choix: ");
        scanf("%d", &niveau);
        int longueurMinl, longueurMaxl;
        if (niveau == 0) {
            longueurMinl = 3;
            longueurMaxl = 4;
        } else if (niveau == 1) {
            longueurMinl = 5;
            longueurMaxl = 6;
        }
    }
}

```

Figure 24 partie 1 de la fonction jouerDeuxJoueurs

```

    longueurMini = 5;
    longueurMaxi = 6;
} else if (niveau == 2) {
    longueurMini = 7;
    longueurMaxi = 8;
} else {
    printf("Choix de niveau non valide. Veuillez choisir 0 pour facile, 1 pour normal, ou 2 pour difficile.\n");
    return;
}
int motsFacilel = 0, motsNormal = 0, motsDifficilel = 0;
rewind(fichier);
while (fscanf(fichier, "%s", motl) != EOF) {
    int motLengthl = strlen(motl);
    if (motLengthl >= 3 && motLengthl <= 4) {
        motsFacilel++;
    } else if (motLengthl >= 5 && motLengthl <= 6) {
        motsNormal++;
    } else if (motLengthl >= 7 && motLengthl <= 8) {
        motsDifficilel++;
    }
}
if (motsFacilel == 0 || motsNormal == 0 || motsDifficilel == 0) {
    printf("Le fichier ne contient pas suffisamment de mots de chaque niveau. L'administrateur doit ajouter des mots de chaque niveau.\n");
    return;
}
do {
    rewind(fichier);
    int indiceAleatoirel = rand() % NbMotl;
    for (int i = 0; i <= indiceAleatoirel; i++) {
        fscanf(fichier, "%s", motl);
    }
} while (strlen(motl) < longueurMini || strlen(motl) > longueurMaxi);
char tab2_l[20];
strcpy(tab2_l, motl);
int LONGUEUR_MOTl = strlen(motl);
char *motSecretl = tab2_l;

```

Figure 25 partie 2 de la fonction jouerDeuxJoueurs

```

char lettreDevineel[LONGUEUR_MOTl];
memset(lettreDevineel, 0, sizeof(lettreDevineel));

    printf("\n Joueur, voici le mot à deviner :%c", motSecretl[0] );
    for (int f = 1; f < LONGUEUR_MOTl; f++) {
        printf("-");
    }

while (1) {
    if (essaisl > NB_ESSAIS_MAX) { //si il atteint le nombre d'essais max
        printf("\n fin zouz khasriin :( !!!\n"); //message qu'il ont perdu

        break; //break: fin de jeu pas besoin de continuer
    }
    char motJoueurl[10];
    printf("\nJoueur 1, entrez un mot de %d : ", LONGUEUR_MOTl);
    scanf("%s", motJoueurl);

```

Figure 26 partie 3 de la fonction jouerDeuxJoueurs

```

int tourJoueur = 1; // variable pour suivre le tour du joueur (1 pour Joueur 1, 2 pour Joueur 2)
int essais1 = 0, essais2 = 0; // Variables pour compter le nombre d'essais pour chaque joueur

// Boucle principale du jeu. continue tant que les deux joueurs n'ont pas atteint le nombre maximum d'essais (6)
while (essais1 < 6 && essais2 < 6) {
    if (tourJoueur == 1) { // Si c'est le tour du Joueur 1
        // Joueur 1
        printf("\nJoueur 1, entrez un mot de %d lettres : ", LONGUEUR_MOT1);
        char motJoueur1[10];
        scanf("%s", motJoueur1);

        // Vérification si le mot a la longueur correcte
        if (strlen(motJoueur1) != LONGUEUR_MOT1) {
            printf("Le mot doit avoir %d lettres.\n", LONGUEUR_MOT1);
            continue; // Rejoue la boucle pour le même joueur
        }
        // Vérification des lettres correctes
        int lettresCorrectes1 = 0;
        for (int f = 0; f < LONGUEUR_MOT1; f++) {
            if (motJoueur1[f] == motSecret1[f]) {
                lettresCorrectes1++;
            }
        }
        // Si toutes les lettres sont correctes, le Joueur 1 a trouvé le mot
        if (lettresCorrectes1 == LONGUEUR_MOT1) {
            printf("Mabrouk, Joueur 1 a trouvé le mot \"%s\" après %d essais.\n", motSecret1, essais1 + 1);
            *scoreJoueur1 += essais1 + 1; // Met à jour le score du Joueur 1
            supprimerMot(choisirThemeFichier(theme), motSecret1); // Supprime le mot trouvé du fichier
            printf("Score après la partie %d - Joueur 1 : %d\n", i + 1, (1000 / (essais1 + 1)));
            printf("\n*****\n");
            break; // Sort de la boucle
        } else {
            printf("Essai incorrect !\n");
        }
        essais1++; // Incrémente le nombre d'essais du Joueur 1
    }
}

```

Figure 27 partie 4 de la fonction jouerDeuxJoueurs

```

} else { //Joueur 2
    printf("\nJoueur 2, entrez un mot de %d lettres : ", LONGUEUR_MOT1);
    char motJoueur2[10];
    scanf("%s", motJoueur2);

    if (strlen(motJoueur2) != LONGUEUR_MOT1) // Vérification si le mot a la longueur correcte
    {
        printf("Le mot doit avoir %d lettres.\n", LONGUEUR_MOT1);
        continue; // Rejoue la boucle pour le même joueur
    }
    // Vérification des lettres correctes
    int lettresCorrectes2 = 0;
    for (int f = 0; f < LONGUEUR_MOT1; f++) {
        if (motJoueur2[f] == motSecret1[f]) {
            lettresCorrectes2++;
        }
    }
    // Si toutes les lettres sont correctes, le Joueur 2 a trouvé le mot
    if (lettresCorrectes2 == LONGUEUR_MOT1) {
        printf("Mabrouk, Joueur 2 a trouvé le mot \"%s\" après %d essais.\n", motSecret1, essais2 + 1);
        *scoreJoueur2 += essais2 + 1; // Met à jour le score du Joueur 2
        supprimerMot(choisirThemeFichier(theme), motSecret1); // Supprime le mot trouvé du fichier
        printf("Score après la partie %d - Joueur 2 : %d\n", i + 1, (1000 / (essais2 + 1)));
        printf("\n*****\n");
        break; // Sort de la boucle
    } else {
        printf("Essai incorrect !\n");
        essais2++; // Incrémente le nombre d'essais du Joueur 2
    }
    tourJoueur = (tourJoueur == 1) ? 2 : 1; // Change le tour du joueur (alterne entre 1 et 2)
}

// Si l'un des joueurs a atteint le nombre maximum d'essais
if (essais1 >= 6 || essais2 >= 6) {
    if (essais1 >= 6) {
        printf("\nJoueur 1 a atteint le nombre maximum d'essais. Le mot était : %s\n", motSecret1);
    }
    if (essais2 >= 6) {
        printf("\nJoueur 2 a atteint le nombre maximum d'essais. Le mot était : %s\n", motSecret1);
    }
}
fclose(fichier); // Ferme le fichier

```

Figure 28 partie 5 de la fonction jouerDeuxJoueurs

e. La fonction admin :

La fonction admin permet à un administrateur d'ajouter ou de supprimer des mots dans un fichier texte. Voici une description du fonctionnement de la fonction :

- L'administrateur choisit entre ajouter (réglage == 0) ou supprimer (réglage == 1) des mots.
- Si l'administrateur choisit d'ajouter des mots, il doit spécifier combien de mots il souhaite ajouter, puis entrer chaque mot individuellement. Les mots sont ajoutés à la fin du fichier.
- Si l'administrateur choisit de supprimer des mots, il voit d'abord la liste des mots existants, puis entre le mot qu'il souhaite supprimer. Le fichier est parcouru, et un fichier temporaire est créé pour stocker tous les mots sauf celui à supprimer. Ensuite, le fichier original est remplacé par le fichier temporaire, réalisant ainsi la suppression du mot sélectionné.

```
#ifndef ADMIN_H
#define ADMIN_H

// Définition de la fonction admin qui permet d'ajouter ou de supprimer des mots dans un fichier
void admin(FILE *fichier) {
    int n; // Variable pour le nombre de mots à ajouter
    int reglage; // Variable pour le choix d'ajouter (0) ou supprimer (1) des mots
    printf("\n*****\n");
    // Demande à l'administrateur de choisir entre ajouter ou supprimer des mots
    printf("Choisissez 0 pour ajouter ou 1 pour supprimer : ");
    scanf("%d", &reglage);
    // Si l'administrateur choisit d'ajouter des mots
    if (reglage == 0) {
        // Demande combien de mots l'administrateur souhaite ajouter
        printf("Combien de mots souhaitez-vous ajouter ? : ");
        scanf("%d", &n);
        // Ouverture du fichier en mode ajout
        fichier = fopen("my_file.txt", "a");
        // Boucle pour entrer chaque mot individuellement
        for (int h = 0; h < n; h++) {
            char motAdmin[10];
            // Demande à l'administrateur d'entrer un mot
            printf("Mode administrateur - Ajouter un mot : ");
            scanf("%s", motAdmin);
            // Écriture du mot dans le fichier
            fprintf(fichier, "%s\n", motAdmin);
        }
        // Fermeture du fichier
        fclose(fichier);
        // Affichage d'un message de succès
        printf("Mots ajoutés avec succès.\n");
    }
}
```

Figure 29 partie 1 de la fonction admin

```

printf("Liste des mots existants :\n");// Affiche la liste des mots existants
char motLecture[50];
rewind(fichier);
// Affiche chaque mot du fichier
while (fscanf(fichier, "%s", motLecture) != EOF) {
    printf("%s\n", motLecture);}
// Demande à l'administrateur d'entrer le mot à supprimer
printf("Entrez le mot que vous souhaitez supprimer : ");
char motASupprimer[50];
scanf("%s", motASupprimer);
// Ouverture d'un fichier temporaire en mode écriture
FILE *fichierTemp = fopen("my_file_temp.txt", "w");
if (fichierTemp != NULL) {
    // Réinitialise la position de lecture dans le fichier original
    rewind(fichier);
    // Copie les mots non supprimés dans le fichier temporaire
    while (fscanf(fichier, "%s", motLecture) != EOF) {
        if (strcmp(motLecture, motASupprimer) != 0) {
            fprintf(fichierTemp, "%s\n", motLecture);}}
    fclose(fichier);// Fermeture des fichiers
    fclose(fichierTemp);
    // Suppression du fichier original
    remove("my_file.txt");
    // Renomme le fichier temporaire pour avoir le nom du fichier original
    rename("my_file_temp.txt", "my_file.txt");
    // Affichage d'un message de succès
    printf("Mot supprimé avec succès.\n");
    }}}
endif

```

Figure 30 partie 2 de la fonction admin

f.La fonction afficheRésultat :

La fonction 'afficherResultat' compare les lettres du mot proposé par le joueur avec celles du mot secret. Elle utilise des couleurs pour indiquer :

Vert : une lettre correctement placée dans le mot secret.

Rouge : une lettre incorrecte ou absente du mot secret.

Bleu : une lettre correcte mais mal placée dans le mot secret.

```

#define COULEUR_H
#define ANSI_COLOR_RESET "\x1b[0m"
#define ANSI_COLOR_RED "\x1b[31m"
#define ANSI_COLOR_GREEN "\x1b[32m"
#define ANSI_COLOR_BLUE "\x1b[34m"
void afficheResultat(const char *motJoueur, const char *motSecret) { //la fonction afficheResultat a comme arguments deux tableau motJoueur et motSecret
    int LONGUEUR_MOT = strlen(motSecret); //la constante LONGUEUR_MOT est la taille de motSecret
    int k=0; //initialisation de compteur k qui va calculer le nombre d'occurrence de chaque lettre
    for (int i = 0; i < LONGUEUR_MOT; i++) { //boucle pour verification
        if (motJoueur[i] == motSecret[i]) { //si la lettre i de mot joueur est la meme de mot secret
            printf(ANSI_COLOR_GREEN "%c " ANSI_COLOR_RESET, motJoueur[i]); //la lettre est correcte elle se colore en vert
        }
        else if (strchr(motSecret, motJoueur[i])) { //sinon si la lettre existe mais pas a sa correcte place
            k++; //le compteur d'occurrence ajoute +1
            if (k>1) { //si k est superieur a 1
                printf(ANSI_COLOR_BLUE "%c " ANSI_COLOR_RESET, motJoueur[i]); //la lettre se colore en bleu
            }
            else if (k==1) { //si k est egal a 1
                printf(ANSI_COLOR_GREEN "%c " ANSI_COLOR_RESET, motJoueur[i]); //sinon si elle se repete qu'une seule fois et cette fois elle est correcte
                printf(ANSI_COLOR_GREEN "%c " ANSI_COLOR_RESET, motJoueur[i]); //elle se colore en vert
            }
        }
        else if ((k==1) && (motJoueur[i] != motSecret[i])) { //sinon si la lettre existe une fois et elle a deja pris sa correcte place
            printf(ANSI_COLOR_RED "%c " ANSI_COLOR_RESET, motJoueur[i]); //elle se colore cette fois en rouge
        }
        else { //sinon si elle existe une fois mais pas a sa place
            printf(ANSI_COLOR_BLUE "%c " ANSI_COLOR_RESET, motJoueur[i]); //elle se colore en bleu
        }
    }

    else { //sinon si elle existe une fois mais pas a sa place
        printf(ANSI_COLOR_BLUE "%c " ANSI_COLOR_RESET, motJoueur[i]); //elle se colore en bleu
    }
}

    else { //on sort de la boucle if: elle n'existe meme pas dans mot secret
        printf(ANSI_COLOR_RED "%c " ANSI_COLOR_RESET, motJoueur[i]); //sinon elle se colore en rouge
    }
    printf("\n"); //retour a la ligne
}
#endif

```

Figure 31 fonction afficheResultat

g. La fonction supprimerMot

Cette fonction, 'supprimerMot', ouvre un fichier contenant une liste de mots. Elle parcourt ce fichier pour supprimer le mot une fois utiliser dans une partie et réécrit le fichier sans ce mot.

```

#define SUPPRIMER_H

#include <stdio.h>
#include <stdlib.h>
#include <string.h>

void supprimerMot(const char *nomFichier, const char *motASupprimer) {
    char motCourant[10]; // Variable pour stocker le mot actuellement lu du fichier
    char **motsTemp = NULL; // Pointeur vers un tableau de chaines de caracteres temporaire
    size_t nbMots = 0; // Nombre de mots actuellement stockés dans motsTemp

    FILE *fichier = fopen(nomFichier, "r"); // Ouvre le fichier en mode lecture
    if (fichier == NULL) { // Verifie si l'ouverture du fichier a échoué
        printf("Erreur lors de l'ouverture du fichier %s.\n", nomFichier);
        return;
    }

    // Lecture des mots du fichier jusqu'à la fin
    while (fscanf(fichier, "%99s", motCourant) != EOF) { // Lit un mot du fichier
        if (strcmp(motCourant, motASupprimer) != 0) { // Verifie si le mot lu n'est pas celui à supprimer
            motsTemp = realloc(motsTemp, (nbMots + 1) * sizeof(char *)); // Réalloue de la mémoire pour motsTemp
            motsTemp[nbMots] = strdup(motCourant); // Copie le mot lu dans motsTemp
            nbMots++; // Incrémente le nombre de mots stockés
        }
    }
}

```

Figure 32 partie 1 de La fonction supprimerMot


```

fclose(fichier); // ferme le fichier apres lecture

fichier = fopen(nomFichier, "w"); // Réouvre le fichier en mode écriture
if (fichier == NULL) { // Vérifie si l'ouverture du fichier a échoué
    printf("Erreur lors de la réouverture du fichier %s.\n", nomFichier);
    // Libération de la mémoire allouée pour motsTemp
    for (size_t i = 0; i < nbMots; i++) {
        free(motsTemp[i]);
    }
    free(motsTemp);
    return;
}

// Écriture des mots restants dans le fichier
for (size_t i = 0; i < nbMots; i++) {
    fprintf(fichier, "%s\n", motsTemp[i]); // Écrit un mot dans le fichier
    free(motsTemp[i]); // Libère la mémoire allouée pour le mot actuel
}
free(motsTemp); // Libère la mémoire allouée pour motsTemp
fclose(fichier); // Ferme le fichier après écriture
}
#endif

```

Figure 33 partie 2 de La fonction supprimerMot

h.La fonction main

Le programme est un jeu de mots appelé "Motus Btounsi" implémenté en langage C. Le jeu offre trois thèmes différents (Choufli Hal, Labsa Tounsiya, Mekla Tounsiya) que l'utilisateur peut choisir. La boucle principale du jeu permet à l'utilisateur de sélectionner le thème et le mode de jeu (administrateur, solo ou duo). Le mode administrateur permet de gérer les thèmes, tandis que les modes solo et duo permettent de jouer avec ou sans un partenaire. Les scores des joueurs sont suivis pour les modes solo et duo. Le programme utilise des fichiers pour stocker les mots associés à chaque thème. Le code est bien structuré, utilisant des fonctions pour organiser les différentes parties du jeu. Les commentaires ajoutés fournissent des explications supplémentaires pour chaque étape du programme, améliorant ainsi la compréhension du code.

```

// Ouverture du fichier du thème sélectionné
FILE *fichierTheme = fopen(choisirThemeFichier(theme), "r");

// Vérification de l'ouverture réussie du fichier du thème
if (fichierTheme == NULL) {
    printf("Erreur lors de l'ouverture du fichier de thème.\n");
    return 1;
}

// Scores pour les modes solo et duo
int scoreJoueur1 = 0;
int scoreJoueur2 = 0;

// Menu de sélection du mode
printf("Choisissez le mode (0 pour administrateur, 1 pour jouer seul, 2 pour jouer à deux, -1 pour quitter) : ");
scanf("%d", &mode);

// Vérification si l'utilisateur souhaite quitter
if (mode == -1) {
    break;
}

// Modes de jeu
if (mode == 0) {
    admin(fichierTheme); // Mode administrateur
} else if (mode == 1) {
    jouerSolo(fichierTheme, &scoreJoueur1, theme); // Mode solo
} else if (mode == 2) {
    jouerDeuxJoueurs(fichierTheme, &scoreJoueur1, &scoreJoueur2, theme); // Mode duo
}

```

Figure 34 partie 1 de La fonction main

```

// Ouverture du fichier du thème sélectionné
FILE *fichierTheme = fopen(choisirThemeFichier(theme), "r");

// Vérification de l'ouverture réussie du fichier du thème
if (fichierTheme == NULL) {
    printf("Erreur lors de l'ouverture du fichier de thème.\n");
    return 1;
}

// Scores pour les modes solo et duo
int scoreJoueur1 = 0;
int scoreJoueur2 = 0;

// Menu de sélection du mode
printf("Choisissez le mode (0 pour administrateur, 1 pour jouer seul, 2 pour jouer à deux, -1 pour quitter) : ");
scanf("%d", &mode);

// Vérification si l'utilisateur souhaite quitter
if (mode == -1) {
    break;
}

// Modes de jeu
if (mode == 0) {
    admin(fichierTheme); // Mode administrateur
} else if (mode == 1) {
    jouerSolo(fichierTheme, &scoreJoueur1, theme); // Mode solo
} else if (mode == 2) {
    jouerDeuxJoueurs(fichierTheme, &scoreJoueur1, &scoreJoueur2, theme); // Mode duo
}

```

Figure 35 partie 2 de La fonction main

```

} while (mode != -1); // Continuer jusqu'à ce que l'utilisateur choisisse de quitter

return 0;
}

```

Figure 36 partie 3 de La fonction main

Conclusion

En conclusion de ce chapitre, nous avons démêlé minutieusement chaque fonction du code, offrant ainsi une vision claire de son rôle et de sa contribution à l'ensemble du projet. Cette analyse approfondie jette les bases nécessaires pour la phase suivante du développement du jeu Motus.

Conclusion générale

En conclusion, ce rapport a tracé le parcours de développement de notre jeu Motus, fusionnant habilement plaisir ludique et concentration. Des bases posées dans le premier chapitre aux détails approfondis des fonctions du code dans le deuxième, notre projet reflète notre engagement à créer une expérience de jeu unique, embrassant la culture tunisienne tout en offrant un divertissement stimulant. Ce travail représente non seulement une réalisation technique, mais aussi l'expression de notre passion pour les jeux et notre désir d'apporter une valeur ajoutée à l'expérience des joueurs.

Bibliographie

<http://files.codes-sources.com/fichier.aspx?id=29972&f Source%5CMotus.cpp>

<http://www.gladir.com/CODER/C/toupper.htm>

<http://www.cplusplus.com/reference/clibrary/cctype/isalpha/>

<http://www.cplusplus.com/reference/clibrary/cstdio/fflush/>

<http://www.programmingsimplified.com/c/conio.h/wherex>

<http://www.cplusplus.com/reference/clibrary/cstdio/fgets/>

<http://www.liafa-jussieu.fr/~yunes/cours/systemes/stdlib.html>